

Listas Enlazadas Semana 02

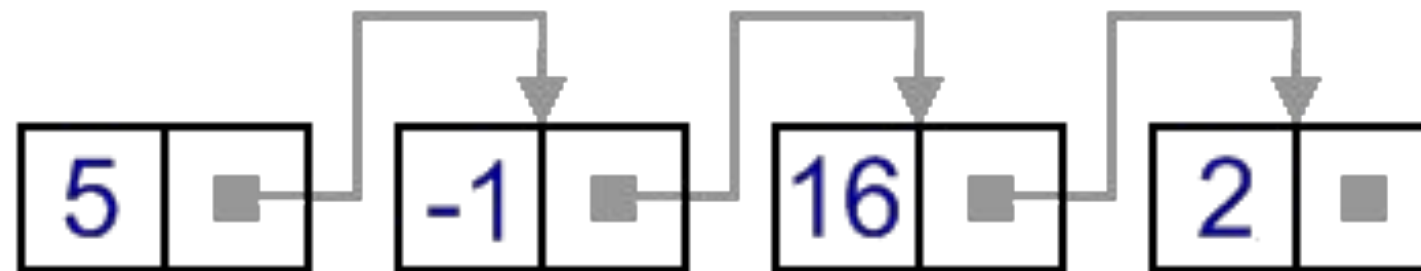
Brenner Ojeda

bojeda@utec.edu.pe



Forward List (Simple Linked List)

Es un contenedor secuencial, el cual permite tener datos en espacios de almacenamiento no relacionados (memoria)



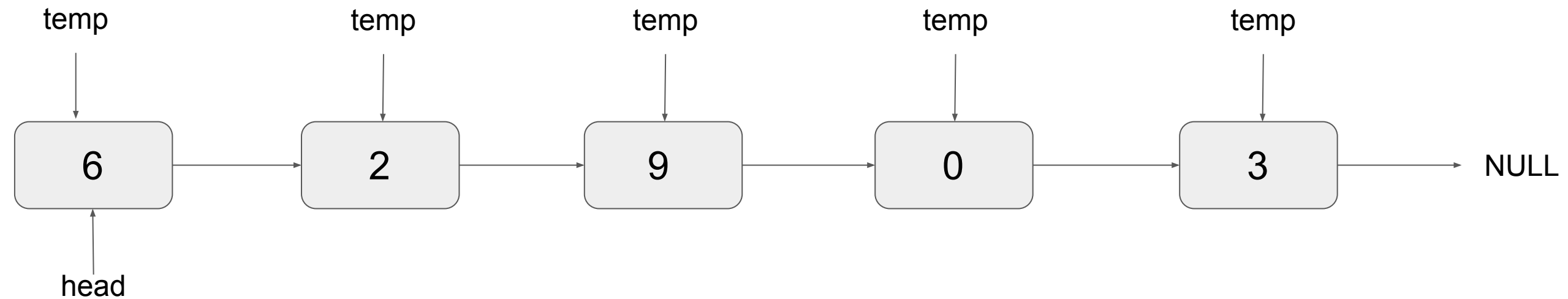
Forward List (Node)

```
struct Node {  
    int data;  
    Node* next;  
};
```

```
class List {  
    private:  
        Node* head;  
  
    ...  
};
```



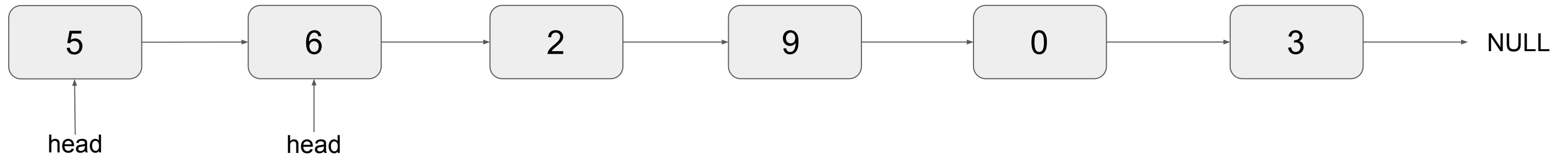
Forward List



```

Nodo* temp = head;
while (temp != NULL) {
    cout<< temp->data;
    temp = temp->next;
}
  
```

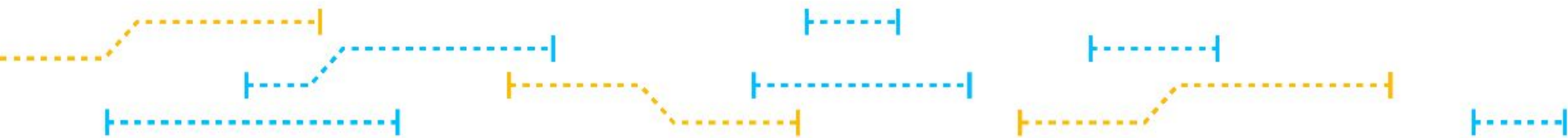
Forward List (push front 5)



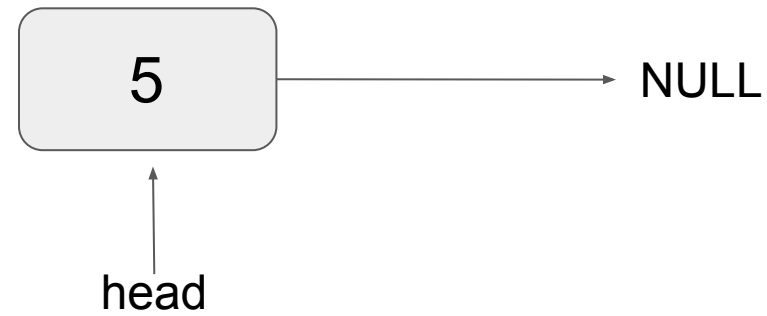
```

Nodo* nodo = new Nodo;
nodo->data = 5;
nodo->next = head;
head = nodo;
  
```

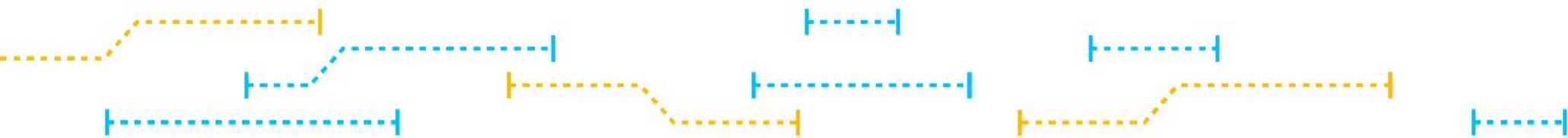
¿Y si la lista está vacía?



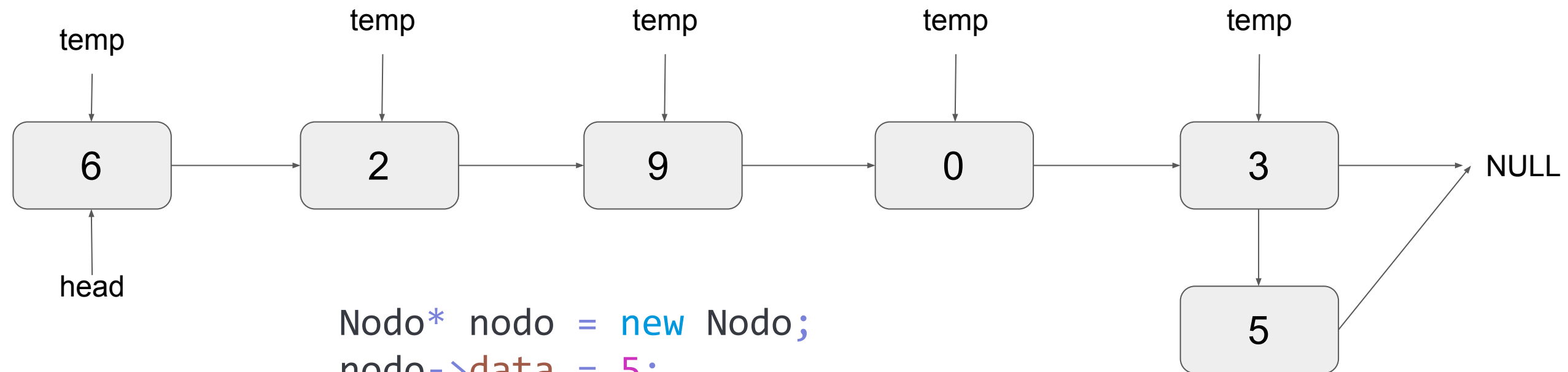
Forward List (push front 5)



```
Nodo* nodo = new Nodo;  
nodo->data = 5;  
nodo->next = head;  
head = nodo;
```



Forward List (push back 5)

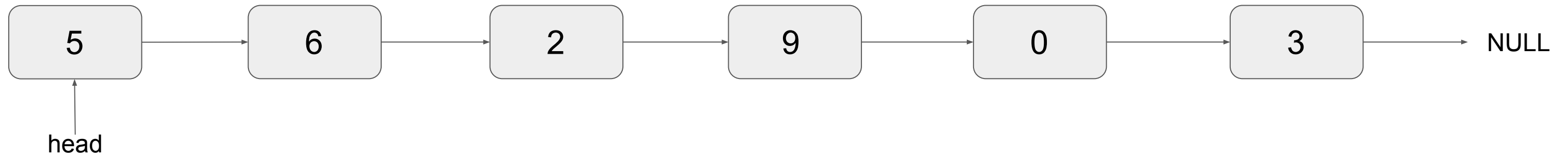


```

Nodo* nodo = new Nodo;
nodo->data = 5;
Nodo* temp = head;
while (temp->next != NULL)
    temp = temp->next;
temp->next = nodo;
nodo->next = NULL;
  
```



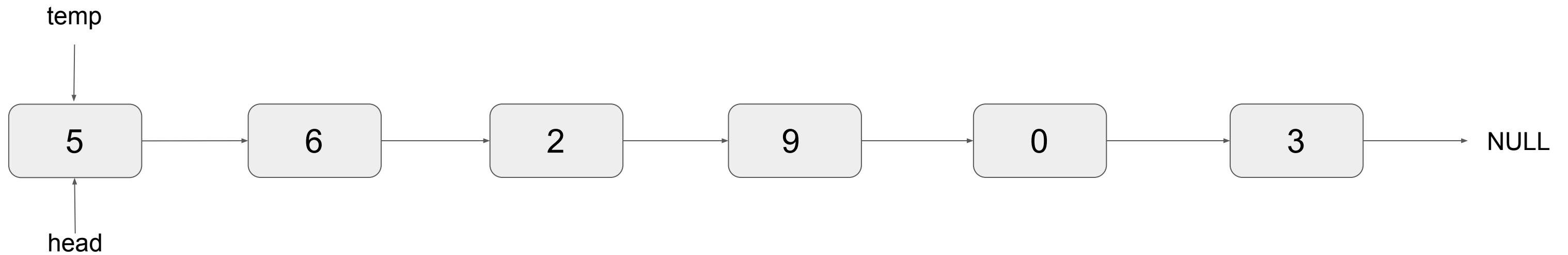
Forward List (pop front)



```
Nodo* temp = head;  
head = head->next;  
delete temp;
```



Forward List (pop back)



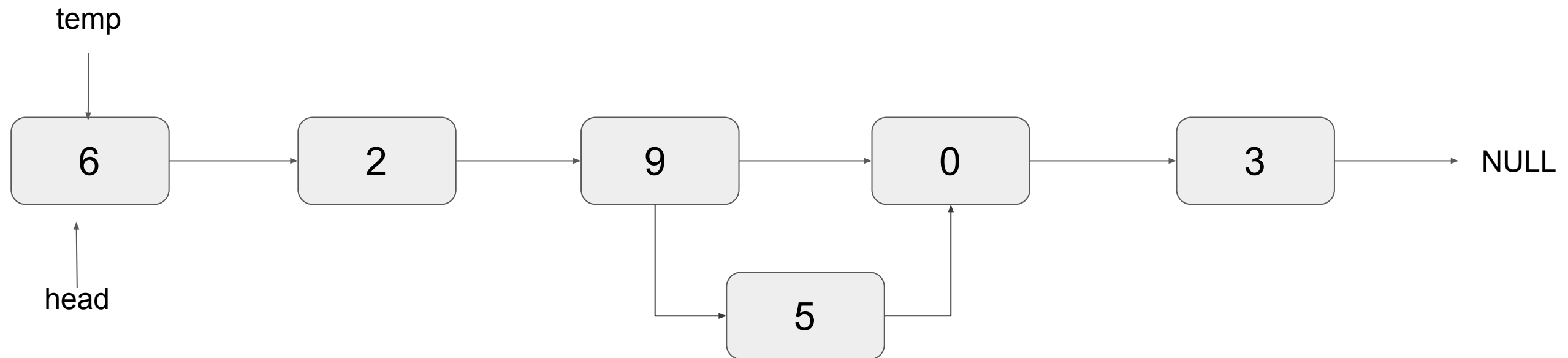
```

if(head->next == NULL)
{
    delete head;
    head = NULL;
}
  
```

```

else
{
    Nodo* temp = head;
    while(temp->next->next != NULL)
        temp = temp->next;
    delete temp->next;
    temp->next = NULL;
}
  
```

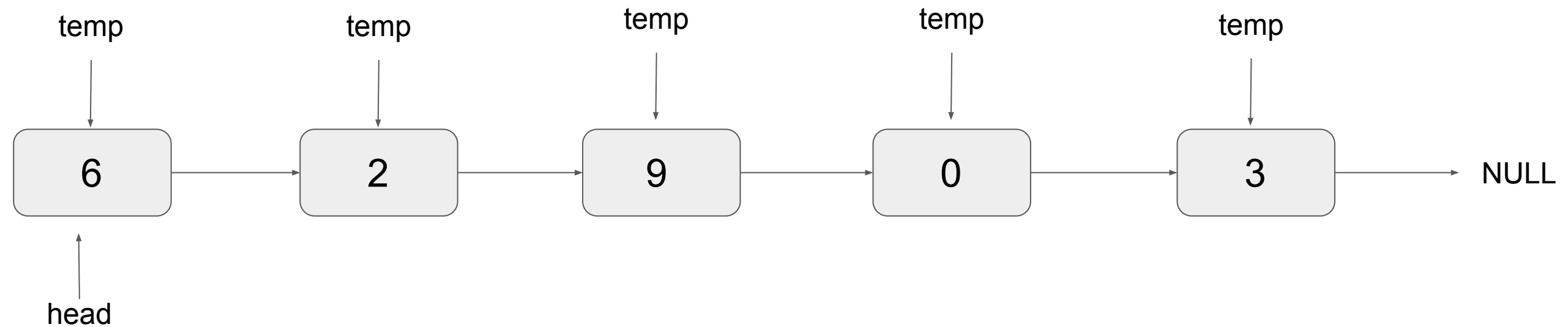
Forward List (insert 5 at location 3)



```

Nodo* nodo = new Nodo(5);
Nodo* temp = head;
int i = 0;
while(i++ < pos - 1) temp = temp->next;
nodo->next = temp->next;
temp->next = nodo;
  
```

Forward List (clear)



```

while(head != NULL)
{
    Nodo* temp = head;
    head = head->next;
    delete temp;

```

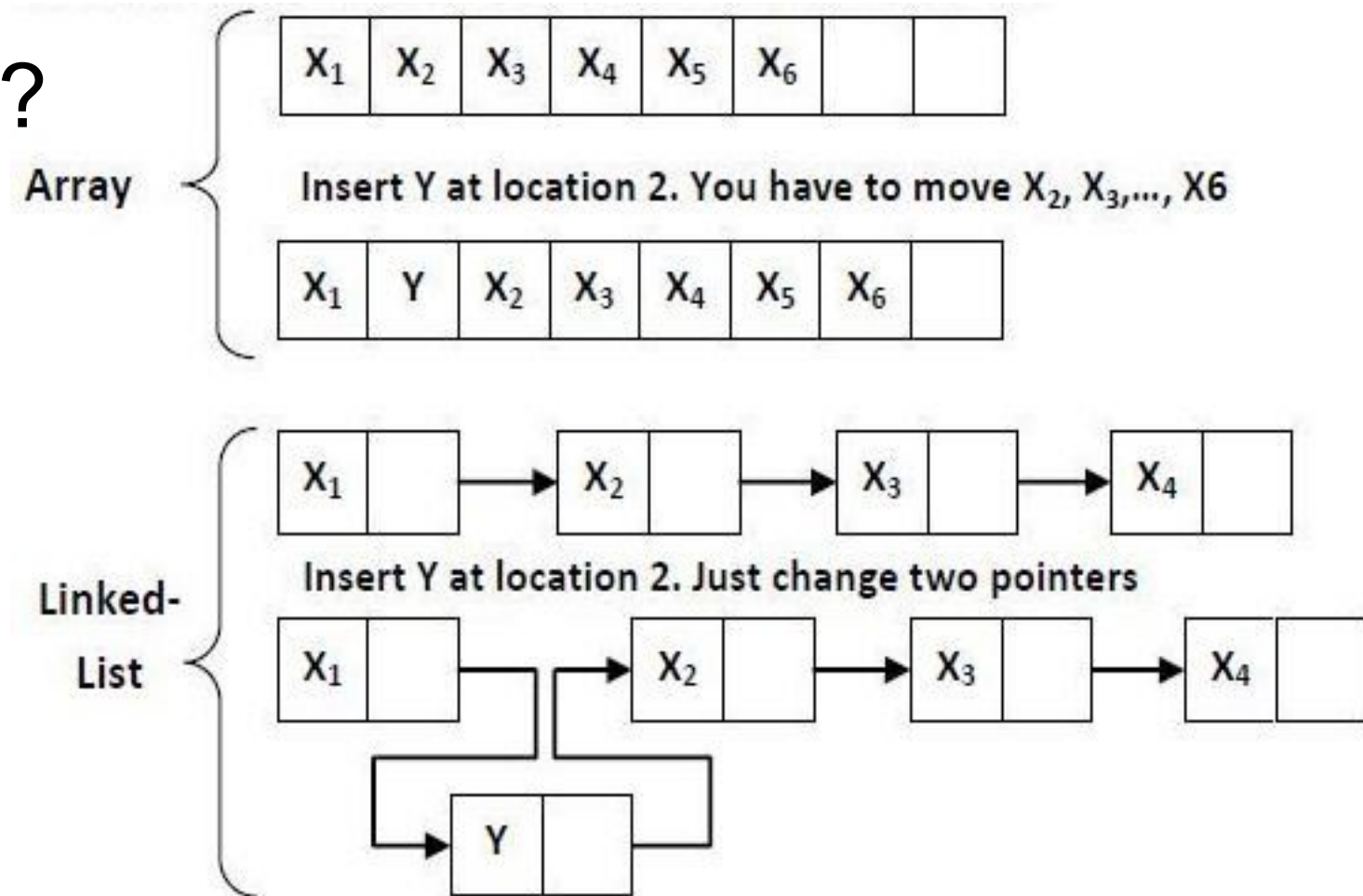
```

}
```

Forward List VS Array

¿Cuál es la diferencia con arreglo?

- Ubicación en la memoria
- Tiempos de acceso
- Tamaño
- Dimensiones



Forward List

1. ¿Cuánto se demora encontrar un elemento al comienzo? ¿Al final? ¿En cualquier posición?

$O(1)$, $O(n)$ y $O(n)$

2. ¿Y para insertar un elemento después del primer nodo? ¿Después del último nodo? ¿Después de cualquier nodo?

$O(1)$, $O(n)$ y $O(n)$

3. ¿Cómo sería el caso 2 pero antes del nodo?

$O(1)$, $O(n)$ y $O(n)$



Forward List

1. ¿Cuánto se demora borrar un elemento al comienzo? ¿Al final? ¿En cualquier posición?

$O(1)$, $O(n)$ y $O(n)$

2. ¿Y para obtener el siguiente elemento de un nodo al comienzo? ¿En cualquier posición?

$O(1)$ y $O(n)$

3. ¿Cómo sería para obtener el nodo anterior al final? ¿En cualquier posición?

$O(n)$ y $O(n)$



Forward List (Homework)

T front(); // Retorna el elemento al comienzo

T back(); // Retorna el elemento al final

void push_front(T); // Agrega un elemento al comienzo

void push_back(T); // Agrega un elemento al final

T pop_front(); // Remueve el elemento al comienzo

T pop_back(); // Remueve el elemento al final

T operator[](int); // Retorna el elemento en la posición indicada

bool empty(); // Retorna si la lista está vacía o no

int size(); // Retorna el tamaño de la lista

void clear(); // Elimina todos los elementos de la lista

void sort(); // Implemente un algoritmo de ordenacion con listas enlazadas)

void reverse(); // Revierte la lista



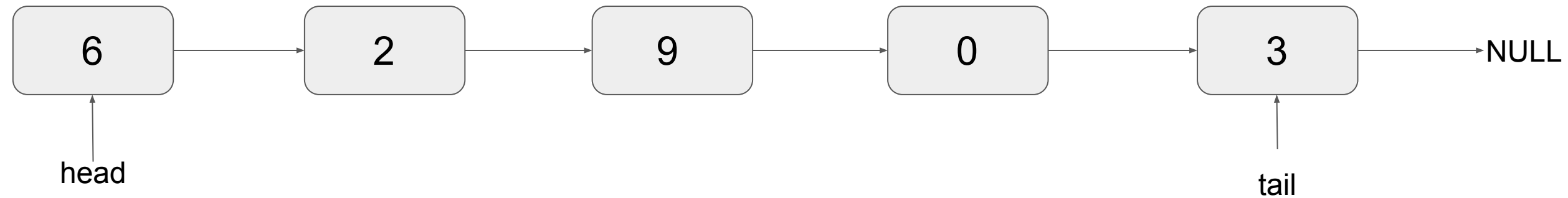
Forward List (Ejercicios)

1. Invertir una lista enlazada
2. **Mezclar dos listas ordenadas en una nueva lista también ordenada $O(n)$.**
3. **Hallar la intersección de dos listas ordenadas en tiempo $O(n)$.**
4. **Implementar el algoritmo de Insertion Sort con listas enlazadas $O(n^2)$.**



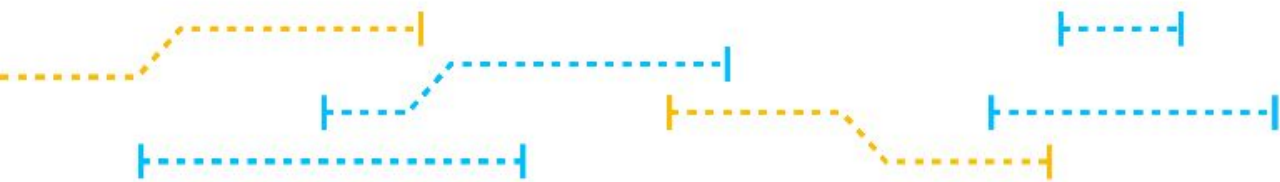
Lists (improves)

¿Cuánto demoraría concatenar dos listas?



¿Cómo borraríamos el último elemento?

¿Cómo imprimiríamos la lista al revés?



Resumen

Data Structure	Operation, Worst Case $O(\cdot)$				
	Container	Static	Dynamic		
	build(X)	get_at(i) set_at(i, x)	insert_first(x) delete_first()	insert_last(x) delete_last()	insert_at(i, x) delete_at(i)
Array	n	1	n	n	n
Linked List	n	n	1	n	n
Dynamic Array	n	1	n	$1_{(a)}$	n